



Algorithmen und Datenstrukturen

Leon Johann Brettin



Algorithmen und Datenstrukturen

12. Bäume



Bäume

- Suchbäume
 - binär
 - Mehrwege
- Ausgeglichene Bäume
 - Rot-Schwarz Bäume
 - realisieren 2-3-4 Bäume
- AVL-Bäume
- Digitale Bäume
 - Tries
 - Heap-Sort



Erinnerung: Sortiervverfahren im Vergleich

Verfahren	Stabilität	Durchschnitt
SelectionSort	Instabil	$\approx n^2/2$
InsertionSort	Stabil	$\approx n^2/4$
BubbleSort	Stabil	$\approx n^2/2$
MergeSort	Stabil	$\approx n \log_2 n$
QuickSort	Instabil	$\approx n \log_2 n$
ShellSort	Instabil	$\approx n^{4/3}$

- Sortieren von Hand: Klausuren nach Matrikelnummern sortieren?
- Lösung: BucketSort oder RadixSort (DistributionSort)
- Voraussetzung: Schlüssel bestehen aus Zeichen eines endlichen Alphabets, für das eine totale Ordnung definiert ist, Schlüssel können „punktweise“ verglichen werden
- Beispiel: natürliche Zahl fester Länge als Schlüssel, betrachte jede Stelle



RadixSort

```
algorithm radixSort(Array von Zahlen x)
    Sortiere n Zahlen mit k Dezimalstellen (m-Stellen)
    for j = 0 to k-1 do
        buckets: array[m] of Integer;
        for i = 1 to n do
            füge x[i] zu bucket [ (x[i] / m^j ) mod m ] hinzu
        od
        konkateniere bucket[] zu x[]
    od
```

- Aufwand?
 - Buckets als Queue:
 - anfügen $O(1)$ (n-mal je äußerer Schleife)
 - Konkatenieren $O(m)$, (1-mal je äußerer Schleife)
 - $O((n+m) * k)$ kann besser sein als $O(n * \log n)$



Sortieren von Lochkarten

- RadixSort
 - Sortiert zunächst nach letzter Dezimalstelle, dann nach vorletzter etc.
 - Wichtig: bei weiteren Schritten Reihenfolge behalten (stabil)!
 - Schon 1887 eingesetzt (Herman Hollerith)

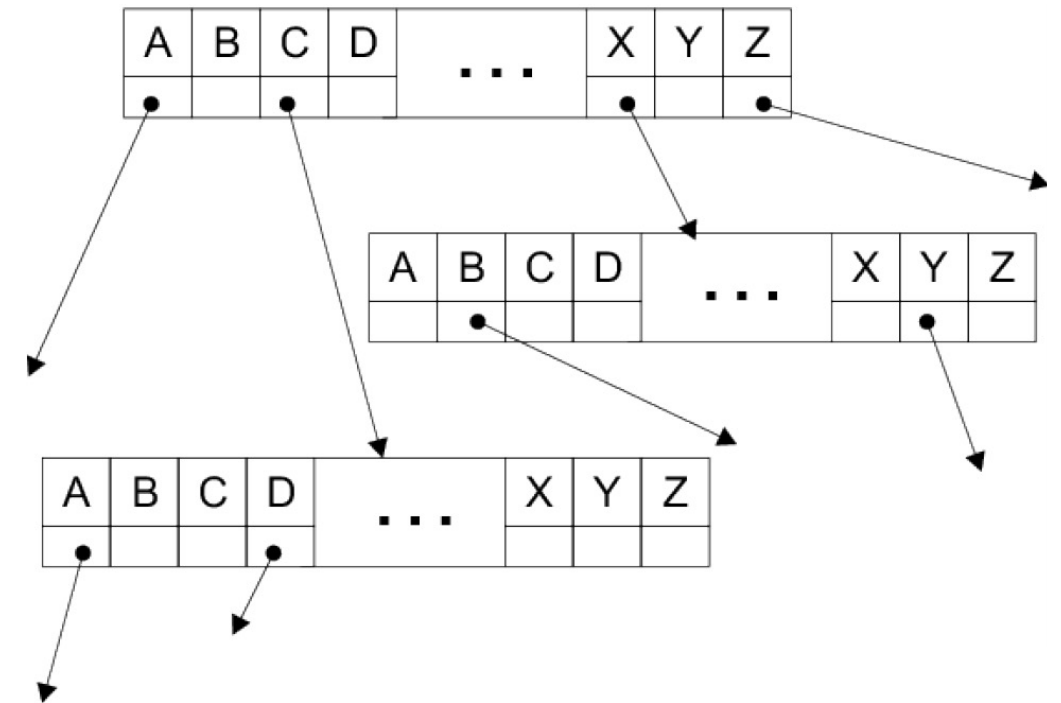


Digitale Bäume

- Binäre Suche verwandt mit binären Bäumen
- BucketSort verwandt mit digitalen Bäumen
- Digitaler Baum: Mehrwegebaum mit festen Verzweigungen
 - von Digitalsystem = Ziffernsystem
 - geeignet nur für Schlüssel, die schon im Stellenwertsystem vorliegen, zum Beispiel Zeichenketten

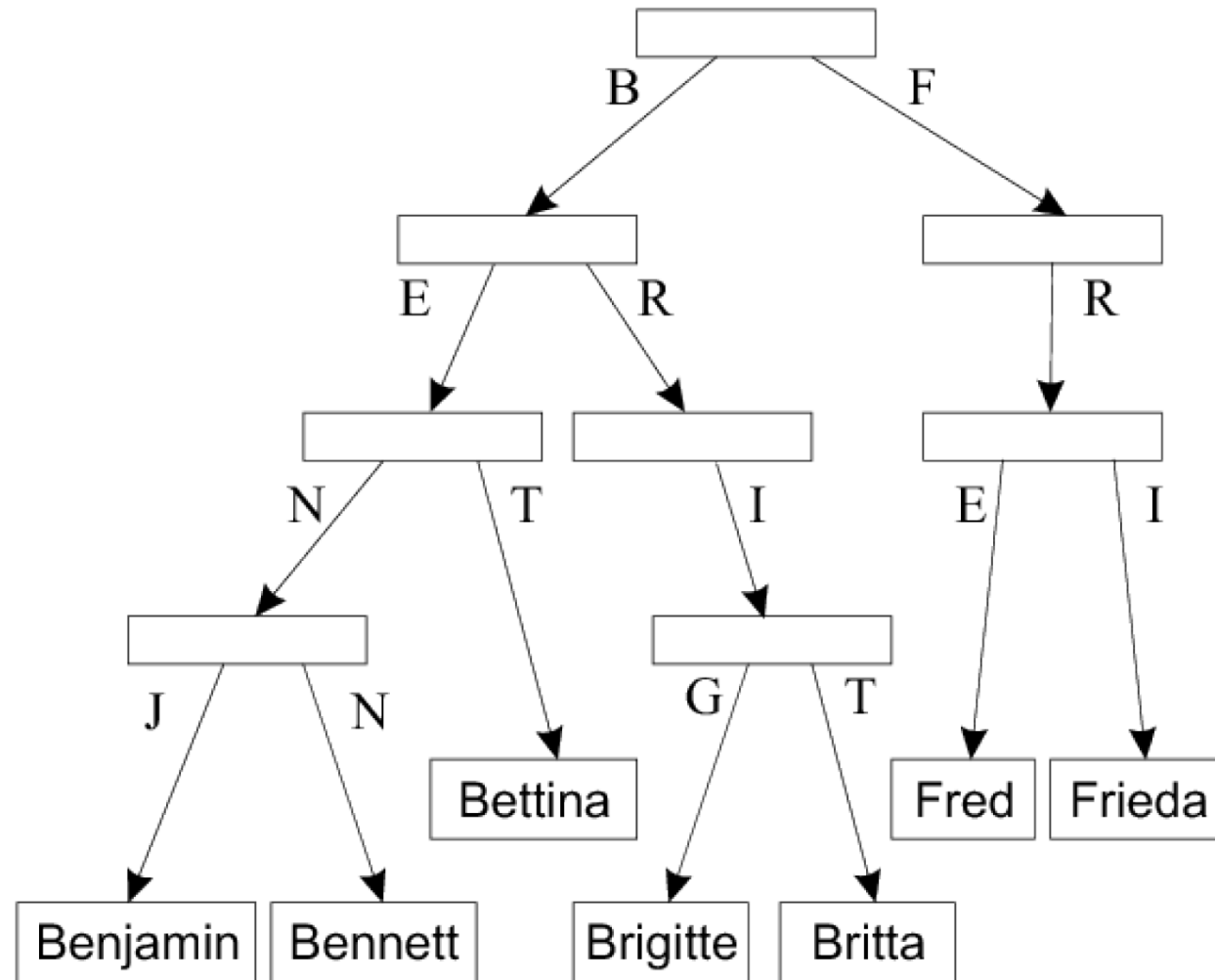
Trie

- Ein **Trie** ist ein digitaler Baum mit Zeichenketten
 - Index, der alle Wörter umfasst (text **retrieval**)
- Knoten: Feld mit der Kardinalität des verwendeten Alphabets
- Buchstaben: 26 Felder,
- Buchstaben selbst nicht abgespeichert!





Beispiel



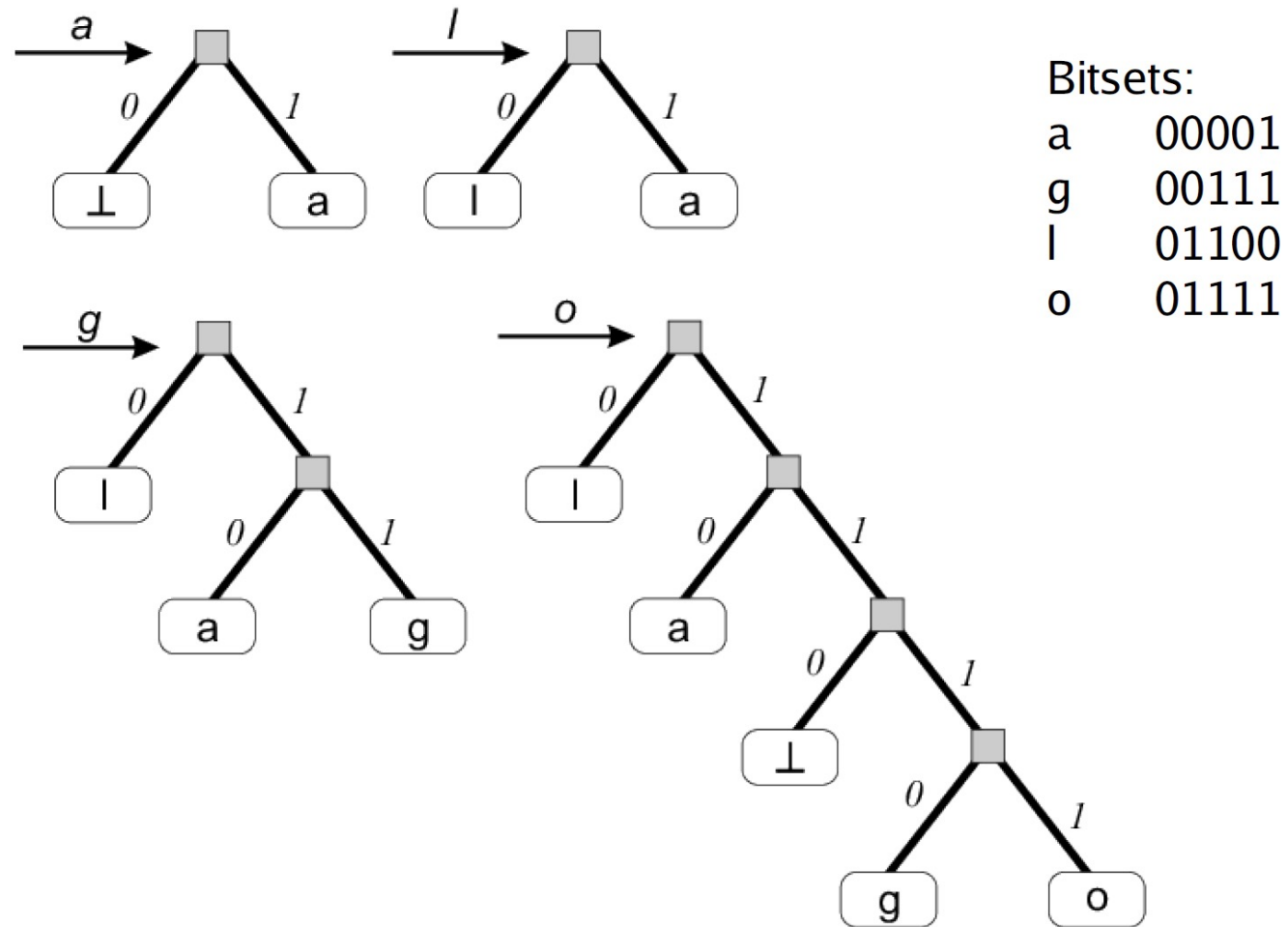


Trie – binärer Trie

- Vorteile:
 - Suchen schneller als binäre Bäume:
 - in jedem Schritt nur ein Zeichen vergleichen.
 - Binärbaum: Alle Zeichen vergleichen, bis Nichtübereinstimmung, und das in jedem Schritt!
- Nachteile
 - Viele Kombinationen von Buchstaben ungenutzt
 - XX... QZ...
- Binärer Trie
 - Schlüssel als Binärzahl auffassen,
 - je nach Verteilung der Daten von vorne oder hinten anfangen!



Beispiel: Schlüssel sind char





Implementierung Trie, Klasse Node

```
abstract class Node {  
    public abstract Node getChild(boolean b);  
    public abstract void setChild(boolean b, Node n);  
}  
class InnerNode extends Node {  
    protected Node child1, child2;  
    public Node getChild(boolean b) {  
        return b ? child1 : child2;  
    }  
    public void setChild(boolean b, Node n) {  
        if (b) {  
            child1 = n;  
        } else {  
            child2 = n;  
        }  
    }  
}  
class Leaf extends Node {  
    protected final char value;  
    public Leaf(char c) {  
        value = c;  
    }  
    public char getKey() {  
        return value;  
    }  
    public Node getChild(boolean b) {  
        throw new RuntimeException("getChild() not allowed in leaf");  
    }  
    public void setChild(boolean b, Node n) {  
        throw new RuntimeException("setChild() not allowed in leaf");  
    }  
}
```





Implementierung Klasse Trie

```
public class Trie {  
    Node root = null;  
  
    public Trie() {  
    }  
  
    private static final boolean getBit(char c, int pos) {  
        return (c & (1 << pos)) != 0;  
    }  
    ...  
}
```





Implementierung Klasse Trie



```
public void insertKey(char c) {
    Leaf leaf = new Leaf(c);
    int pos = 0;
    if (root == null) { // Spezialfall: leerer Trie
        root = new InnerNode();
        root.setChild(getBit(c, pos), leaf);
    } else {
        Node node = root;
        while (node != null) {
            Node parent = node;
            node = node.getChild(getBit(c, pos));
            if (node == null) { // leerer Blattknoten, direkt einfügen
                parent.setChild(getBit(c, pos), leaf);
            } else if (node instanceof Leaf) {
                Leaf oldLeaf = (Leaf) node;
                char oldKey = oldLeaf.getKey();
                if (oldKey == c) {
                    break; // Doublette, nicht einfügen
                }
                int j; // erste abweichende Bitposition suchen
                for (j = pos; getBit(oldKey, j) == getBit(c, j); j++) {
                    Node newNode = new InnerNode(); // Auf dem Weg innere Knoten erzeugen
                    parent.setChild(getBit(oldKey, j), newNode);
                    parent = newNode;
                }
                // neuen Knoten einfügen
                parent.setChild(getBit(oldKey, j), oldLeaf);
                parent.setChild(getBit(c, j), leaf);
                break;
            } else { pos++; }
        }
    }
}
```



Implementierung Trie

```
boolean lookupKey(char c) {  
    Node node = root;  
    int pos = 0;  
    while (node != null) {  
        node = node.getChild(getBit(c, pos++));  
        if (node == null)  
            break;  
        if (node instanceof Leaf) {  
            Leaf leaf = (Leaf) node;  
            return leaf.getKey() == c;  
        }  
    }  
    return false;  
}
```





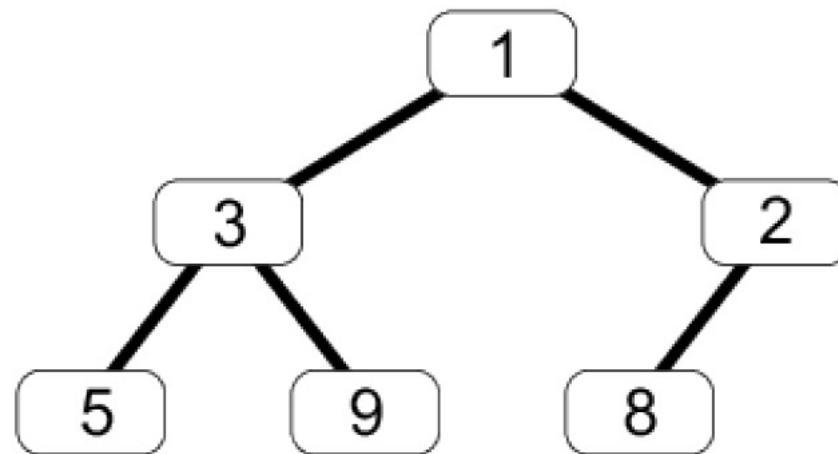
Sortieren mit Bäumen: HeapSort

- **Priority Queue** ist eine abstrakte Datenstruktur, die folgende Operationen effizient erlaubt:
 - `Object getMinimum()`: liefert und entfernt das kleinste Element der Queue.
 - `void add(Object o)`: fügt ein Element in die Queue ein.
 - oder alternativ
 - `PriorityQueue(Object[] array)`: erzeugt aus dem Feld eine Prioritätswarteschlange.
- Wie implementieren?
 - Binärbaum (Rot-Schwarz)
 - `getMinimum()` ist $O(\log n)$
 - `add()` ist $O(\log n)$



Heap

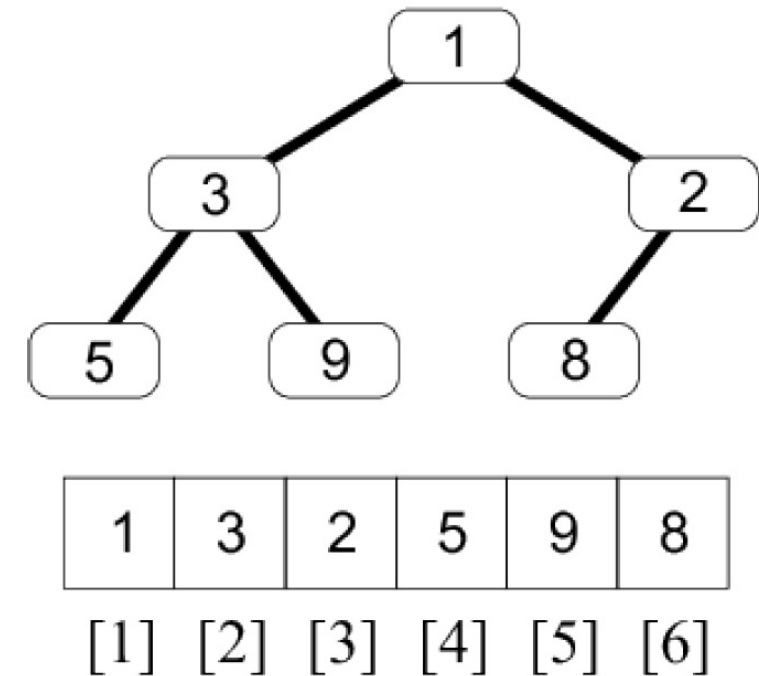
- Ein Heap ist ein Binärbaum, der folgende Eigenschaften hat
 - Der Baum ist vollständig, d.h. die Blattebene ist von links nach rechts gefüllt
 - Der Schlüssel eines jeden Knotens ist kleiner (oder gleich) als die Schlüssel seiner Kinder – **Heap-Eigenschaft**
- Die Wurzel ist immer das kleinste Element





Heap als Feld

- Wir speichern den Heap in einem Feld.
- Knoten i hat Kinder
 - $2*i$ und
 - $2*i+1$
 - Bei Indizierung ab 1



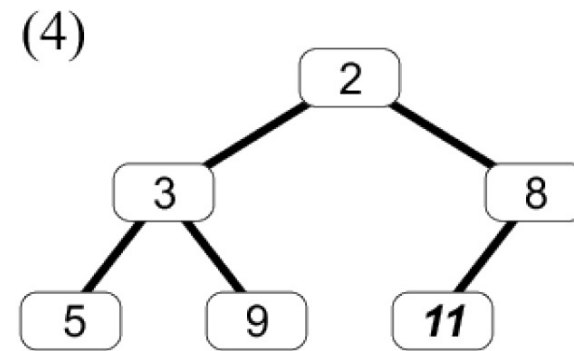
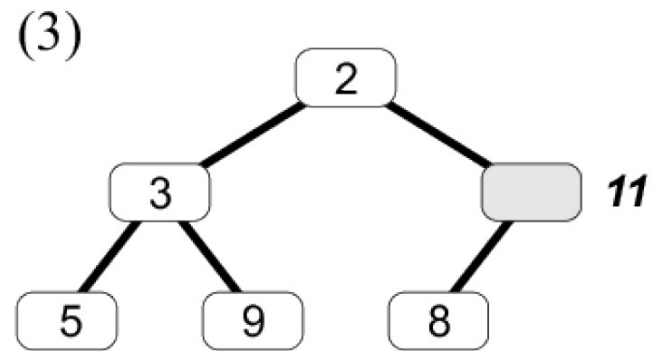
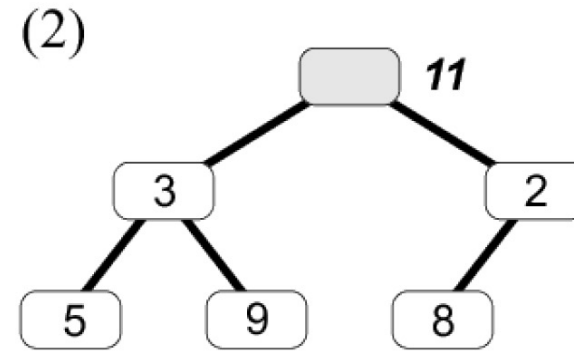
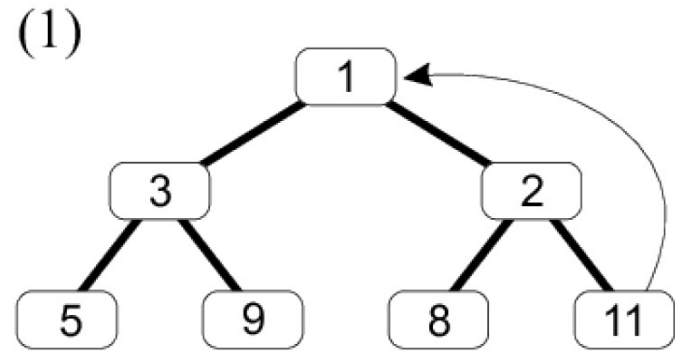


getMinimum()

- `getMinimum()` liefert die Wurzel `arr[1]`.
- Es entsteht ein „Loch“
- Wird gefüllt durch letztes Element im Heap
 - „rechtestes“ Element in der Blattebene
- Heap-Eigenschaft muss wieder hergestellt werden: „reHeap“

```
algorithm reHeap(F, start, end)
    Eingabe: F Feld,
    int start, end Grenzen des Heap-Bereichs.
    node = F[start];
    while node.hasChildren and not
        (node.value < child1.value and
         node.value < child2.value) do
        // kleineres Kind nach oben tauschen
        if child1.value < child2.value then
            swap(node, child1)
        else
            swap(node, child2)
        fi
    od
```

Beispiel





reheap() im Array

```
algorithm reHeap(F, start, end)
  Eingabe: F Feld,
  int start, end Grenzen des Heap-Bereiches.
  node = F[start];
  while node.hasChildren and not
    (node.value < child1.value and
     node.value < child2.value) do
    // kleineres Kind nach oben tauschen
    if child1.value < child2.value then
      swap(node, child1)
    else
      swap(node, child2)
    fi
  od
```

```
algorithm reHeap(F, start, end)
  Eingabe: F Feld,
  int start, end Grenzen des Heap-Bereiches.
  i = start;
  while 2*i < end and not
    (F[i] < F[2*i] and F[i] < F[2*i+1]) do
    // kleineres Kind nach oben tauschen
    if F[2*i] < F[2*i+1] then
      swap(F, i, 2*i); i = 2*i;
    else
      swap(F, i, 2*i+1); i = 2*i+1;
    fi
  od
```



Sortieren mit HeapSort

- Sortieren mit Heap
 - ähnlich wie SelectionSort
 - Selection geht schnell: Wurzel nehmen.
 - bei MinHeap: umgekehrte Sortierung
 - Reheap: $O(\log n)$

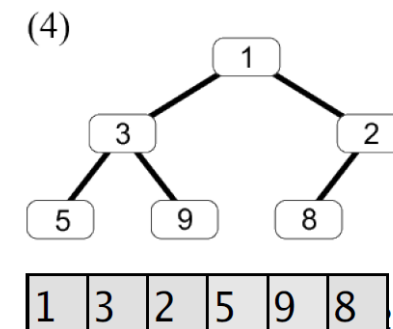
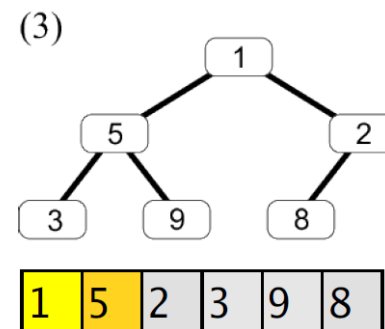
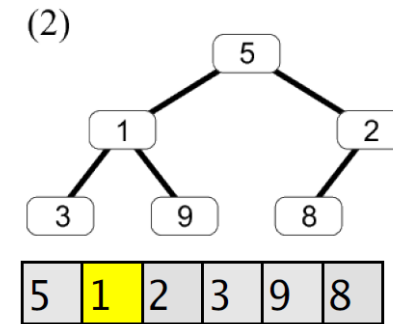
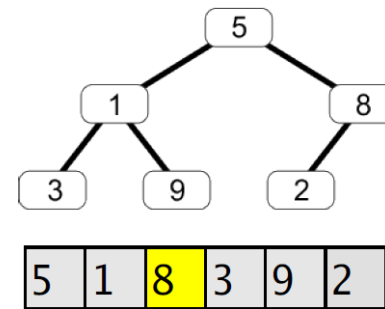
```
algorithm HeapSort (F)
  Eingabe: zu sortierende Folge F
  Überführe F in einen Heap;
  k := n; // Position des letzten Elements
  while k > 1 do
    swap(F[1], F[k]);
    reheap(F, 1, k-1);
    k := k-1;
  od
```



Heap herstellen

- Heap herstellen: von unten nach oben.
- Blätter haben immer Heap-Eigenschaft, deshalb $k := n/2$

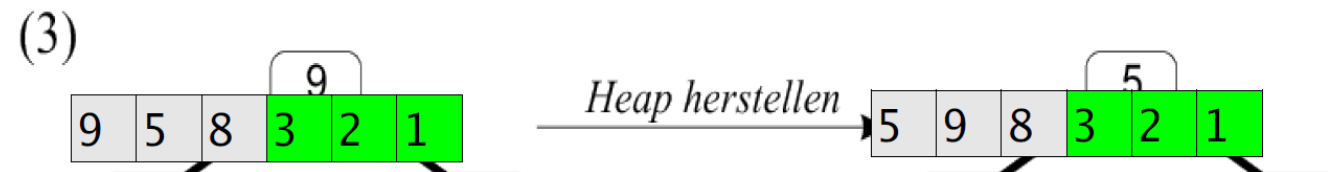
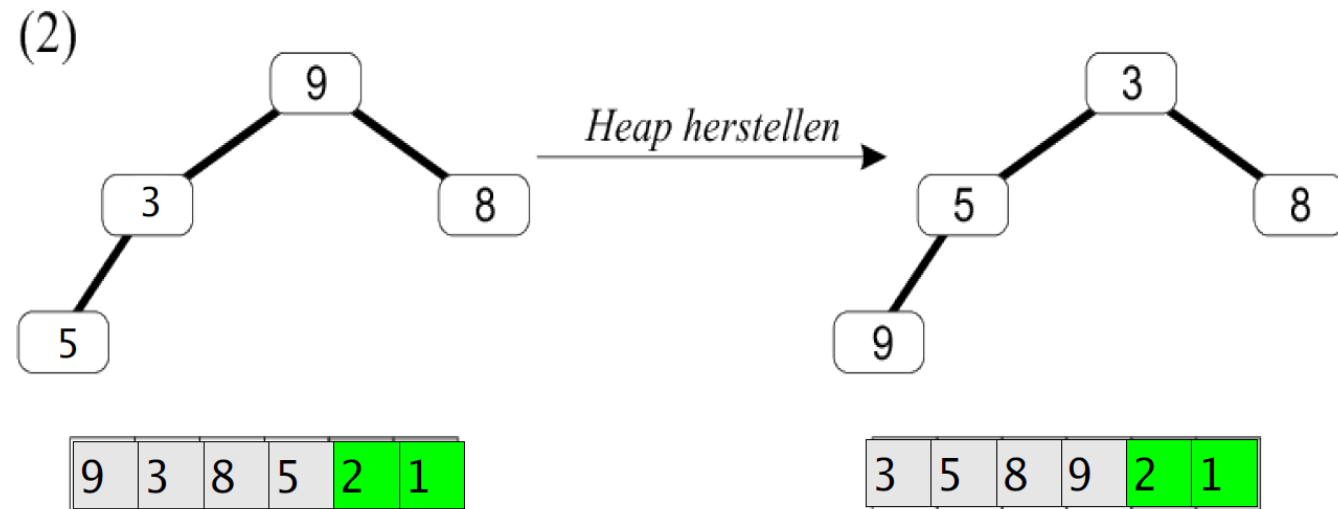
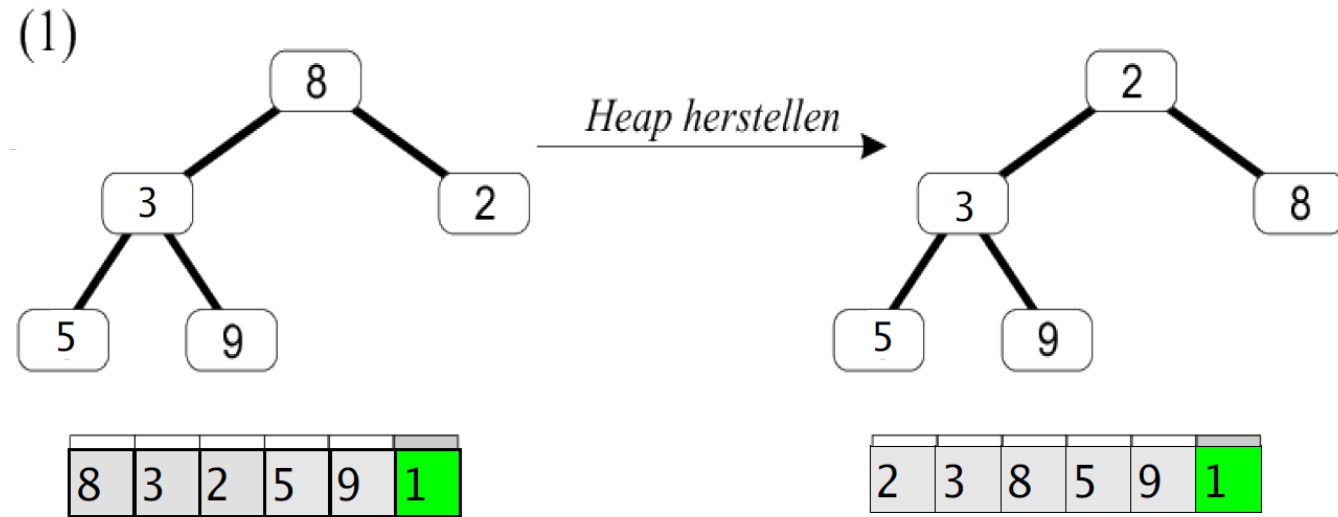
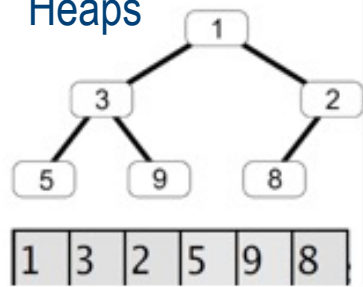
```
algorithm makeHeap (F)
  Eingabe: zu sortierende Folge F
  // Überführt F in einen Heap;
  for k := n/2 downto 1 do
    reheap(F, k, n);
  od
```





Sortieren

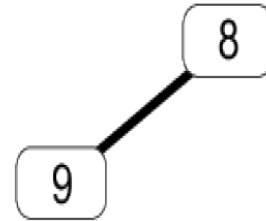
Ausgangssituation
nach Erstellen des
Heaps





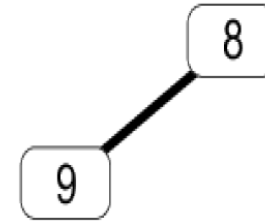
Sortieren

(4)



8	9	5	3	2	1

Heap herstellen



8	9	5	3	2	1

(5)



9	8	5	3	2	1

Heap herstellen



9	8	5	3	2	1



HeapSort

- Komplexität:
 - Herstellen des Heap: $n/2 * \log_2 n = O(n * \log n)$
 - Sortieren durch Abbau: $n * \log_2 n = O(n * \log n)$
- Im schlimmsten Fall:
 - $O(n * \log n)$
 - besser als Quicksort,
 - vergleichbar mit MergeSort.
- InPlace
- nicht stabil!
- Durchschnitt: langsamer als Quicksort!